

7 — Local Development Guide

Audience: Developer **Requirements:** Node.js 18+, Firebase CLI, Python 3.8+

Prerequisites

Tool	Install
Node.js 18+	nodejs.org
Firebase CLI	<code>npm install -g firebase-tools</code>
Python 3.8+	python.org

Step 1 — Clone & Install Dependencies

```
# Clone the repository
git clone <your-repo-url>
cd pumping-station-iot

# Install backend Cloud Functions dependencies
cd backend/functions && npm install && cd ../..

# Install frontend dependencies
cd frontend && npm install && cd ..
```

Step 2 — Configure Environment Variables

Frontend

```
# Copy the example and fill in your Firebase project credentials
cp frontend/.env.example frontend/.env
```

Edit `frontend/.env` with your Firebase project settings from: **Firebase Console** → **Project Settings** → **Web App**

Simulation Script (optional)

```
# Copy root .env.example
cp .env.example .env
```

For local development, the defaults work out of the box (emulator URL, no auth).

Step 3 — Start Firebase Emulators

```
cd backend
firebase emulators:start
```

Wait until you see: **"All emulators ready!"**

The emulators provide:

Service	Port
Auth	http://127.0.0.1:9099
Functions	http://127.0.0.1:5001
Firestore	http://127.0.0.1:8080
Realtime DB	http://127.0.0.1:9000
Storage	http://127.0.0.1:9199
Emulator UI	http://127.0.0.1:4000

Step 4 — Seed Admin User

In a **new terminal** (keep the emulators running):

```
cd backend/functions

# Seed with default admin@example.com
node seed-users.js

# Or specify a custom admin email
ADMIN_EMAIL=your-email@gmail.com node seed-users.js
```

Step 5 — Start the Frontend Dev Server

```
cd frontend
npm run dev
```

Open **http://localhost:5173** in your browser.

Login

1. Click "**Sign in with Google**"
2. The Firebase Auth Emulator popup will appear
3. Click "**Add new account**"
4. Enter the same email you used in the seed step
5. Click **Sign In**

Note: On localhost, any Google account is automatically promoted to admin.

Step 6 – Run the Station Simulator (Optional)

In a **new terminal**:

```
# Run the simulator (Requires Python 3)
python simulate_station.py --station <YOUR_STATION_ID> --token <YOUR_CUSTOM_TOKEN> --apikey <YOUR_
```

This simulates a single pumping station sending real-time telemetry data to the local RTDB emulator. The dashboard will update in real-time.

Simulation Configuration

You can pass arguments to control its behavior:

- `--station` – The Station ID (must be uppercase)
 - `--token` – The custom token generated from the dashboard
 - `--apikey` – The Web API Key of your Firebase project
 - `--dburl` – (Optional) Target database URL, defaults to the production DB. Use `http://localhost:9000` for the emulator.
-

Project Structure

```

pumping-station-iot/
├── backend/
│   ├── firebase.json          # Firebase emulator + deploy config
│   ├── firestore.rules        # Firestore security rules
│   ├── firestore.indexes.json # Composite indexes
│   ├── database.rules.json    # Realtime Database security rules
│   ├── storage.rules          # Cloud Storage security rules
│   └── functions/
│       ├── src/
│       │   ├── index.js       # Cloud Functions entry point
│       │   ├── alerts.js      # Alert detection on live data
│       │   ├── archive.js     # Archive readings to Firestore
│       │   ├── email.js       # SendGrid email notifications
│       │   ├── offline.js     # Offline station detection
│       │   ├── provision.js   # Device provisioning
│       │   └── purge.js       # Scheduled data cleanup
│       ├── seed-users.js      # Seed admin user for emulator
│       └── seed.js            # Alternative seed script
├── frontend/
│   ├── .env.example          # Firebase config template
│   ├── src/
│   │   ├── firebase.ts       # Firebase initialization
│   │   ├── auth.ts           # Authentication service
│   │   ├── app.ts            # Main app router
│   │   ├── dashboard.ts      # Dashboard view
│   │   ├── station-detail.ts  # Station detail + live chart
│   │   ├── station-mgmt.ts    # Station management (admin)
│   │   ├── user-mgmt.ts      # User management (admin)
│   │   ├── alerts.ts         # Alerts list view
│   │   ├── map.ts            # Map view (Leaflet)
│   │   ├── history.ts        # Historical data + charts
│   │   └── i18n.ts           # Internationalization (EN/DE)
│   └── css/
│       ├── variables.css     # Design tokens
│       └── components.css    # Component styles
├── firmware/                 # ESP32 firmware (PlatformIO)
├── docs/                     # Project documentation
├── simulate_station.py       # Station data simulator
└── .env.example              # Simulator config template

```

Troubleshooting

"Failed to load function" in emulator

The emulators automatically hot-reload when you edit `backend/functions/src/*.js` files. If you see repeated load failures, check for syntax errors in the function files.

"No matching allow statements" in browser console

This means a Firestore security rule is blocking the query. Check that:

1. The emulators loaded the latest `firestore.rules`
2. The user document exists in Firestore (run `seed-users.js`)

Station data not appearing

1. Verify the simulator is running (`python simulate_stations.py`)
2. Check the emulator UI at `http://127.0.0.1:4000` → Database to see if data is being written
3. Ensure the frontend `.env` has the correct `VITE_FIREBASE_PROJECT_ID`

Previous: [Alerts & Scaling Guide](#) ←